

Solver Specification:

highOrderElectroActivationFoamImplicit

A high-order, implicit Picard monodomain solver in OpenFOAM with an Eigen linear-algebra back-end

Pablo

May 21, 2026

Abstract

This report specifies the algorithms implemented in the OpenFOAM solver:

highOrderElectroActivationFoamImplicit.

The solver advances the monodomain equations of cardiac electrophysiology coupled to the ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004) ionic cell model. The diffusion operator is discretised with the cell-centred finite-volume framework of OpenFOAM, optionally upgraded to a Local Regression Estimators (LRE) high-order reconstruction; the ionic source is evaluated either at cell centres or at LRE cell quadrature points; time integration follows a θ -scheme (backward Euler or Crank–Nicolson); the resulting *nonlinear* coupled system is solved with a fixed-point Picard iteration using a fixed number of nonlinear sweeps. In each sweep the ionic ODE is integrated at the current frozen voltage, the ionic source is placed on the right-hand side, and the voltage is updated by solving the unchanged implicit diffusion operator. The sparse linear system is factored or iteratively solved with Eigen (**SparseLU** or **BiCGSTAB+ILUT**); the per-cell ionic ODE uses OpenFOAM’s adaptive RKF45 integrator with configurable tolerances; the TNNP cell model is wrapped in a phase-aware **protectState** guard that enforces physical bounds before every algebraic evaluation; activation times are recorded by linear interpolation across the first threshold crossing and benchmark points are sampled by inverse-distance weighting. The report also discusses the limits of the single-sweep Picard regime used in the reported runs (where the ionic source is effectively lagged by one step and the temporal error is no longer controlled by θ) and the assumptions, scope, and side-effects of the TNNP **protectState** guard, including its interaction with the RKF45 step controller. All algorithmic parameters are exposed through the **constant/spatialIntegrationProperties** and **constant/timeIntegrationProperties** dictionaries, and the companion Python driver **run.convergence.py** generates these dictionaries from a single configuration file. Section 12 collects measured P8 activation times, wall-clock cost and peak memory from four independent result trees of a 2D Niederer 2012 benchmark.

Contents

1	Notation	3
2	Governing equations	3
2.1	Monodomain PDE	3
2.2	Ionic ODE	3
2.3	External stimulus	4
3	Time discretisation: the θ-scheme	4
4	Spatial discretisation	4
4.1	Standard orthogonal FV Laplacian	4
4.2	High-order LRE diffusion	5

4.3	Mass matrices	5
4.4	High-order reconstruction at ionic quadrature points	5
5	Nonlinear solver	5
5.1	Picard iteration	6
5.2	Iteration budget and acceptance	6
6	Linear solver: Eigen back-end	6
7	Ionic ODE integration	6
8	TNNP numerical protection	7
8.1	Master switch and configuration	7
8.2	Phases at which the guard fires	7
8.3	Corrections and their order	7
8.4	Interaction with the RKF45 step controller	8
8.5	What the guard does not protect	8
8.6	Logging and accounting	9
8.7	Defaults vs. recommended settings	9
9	Activation tracking and benchmarks	9
10	Solver flow chart	10
11	Driver script: run_convergence.py	10
11.1	Geometry and parameter sweep	10
11.2	Nonlinear and linear solver	11
11.3	Inner ODE integrator (RKF45)	11
11.4	Post-processing and run-control	11
12	2D Niederer-style benchmark — in-house convergence study	11
12.1	Results for P8 activation	12
12.2	Results for Wall-clock time	12
12.3	Results for Peak RSS	13
12.4	Discussion of the 2D results	14
12.5	Conclusions on the 2D study	14
13	3D Niederer-style benchmark	15
13.1	P8 activation time at $\Delta t = 5 \cdot 10^{-5}$ s	15
13.2	Wall-clock cost per configuration (minutes)	15
13.3	Peak resident set size per configuration (MiB)	16
13.4	Discussion of the 3D results	16
A	Dimension table	16

1 Notation

Symbol	Meaning	Units
$\Omega \subset \mathbb{R}^d$	Spatial domain ($d \in \{2, 3\}$)	m
$\partial\Omega$	Insulated boundary (zero-flux Neumann)	—
$t \in (0, T]$	Time	s
$V_m(\mathbf{x}, t)$	Transmembrane voltage	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	TNNP ionic states	mixed
$I_{\text{ion}}(V_m, \mathbf{s})$	Total ionic current density	V s^{-1}
$I_{\text{ext}}(\mathbf{x}, t)$	External stimulus current density	A m^{-3}
χ	Membrane surface-to-volume ratio	m^{-1}
C_m	Membrane capacitance per unit area	F m^{-2}
$\mathbf{D}(\mathbf{x})$	Bulk conductivity tensor	S m^{-1}
$\theta \in \{0.5, 1\}$	Time-stepping parameter (CN or BE)	—
Δt	Outer time-step length	s
N_c	Number of mesh cells	—
$\mathbf{V}^n \in \mathbb{R}^{N_c}$	Vector of cell-averaged V_m at time t^n	V
M, K, L	Mass, stiffness, scaled diffusion matrix	—
A, B	Implicit LHS / RHS operators of the θ -scheme	—

2 Governing equations

2.1 Monodomain PDE

The bulk voltage is propagated by the standard monodomain reaction–diffusion equation [1, 2]:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}} \quad \text{in } \Omega \times (0, T], \quad (1)$$

subject to insulated boundaries $\mathbf{n} \cdot (\mathbf{D} \nabla V_m) = 0$ on $\partial\Omega \times (0, T]$, and initial condition $V_m(\mathbf{x}, 0) = -84 \text{ mV}$. The conductivity tensor $\mathbf{D}$ is symmetric positive-definite and either uniform or read as a `volTensorField` from disk.

Dividing (1) by χC_m gives the form actually used inside the solver:

$$\frac{\partial V_m}{\partial t} = \mathcal{L} V_m + s(V_m, \mathbf{s}), \quad \mathcal{L} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla \cdot), \quad s = -\frac{I_{\text{ion}}}{\chi C_m} + \frac{I_{\text{ext}}}{\chi C_m}. \quad (2)$$

2.2 Ionic ODE

At every spatial sampling point the TNNP state vector $\mathbf{s} \in \mathbb{R}^{17}$ evolves according to

$$\frac{d\mathbf{s}}{dt} = \mathbf{f}_{\text{TNNP}}(V_m, \mathbf{s}; \text{tissue}), \quad \mathbf{s}(0) = \mathbf{s}_0, \quad (3)$$

where `tissue` selects `epicardialCells`, `mCells` or `endocardialCells`. The 17 tracked states are

$$\mathbf{s} = (V, K_i, Na_i, Ca_i, Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g, Ca_{SR}).$$

The total ionic current returned to the PDE is `Iion.cm` of TNNP [1].

Units and sign convention. The field `Iion.cm` returned by the TNNP wrapper is the per-capacitance ionic current,

$$\text{Iion.cm} = \frac{I_{\text{ion}}^{(\text{memb})}}{C_m} \equiv \frac{1}{\chi C_m} (\chi I_{\text{ion}}^{(\text{memb})}),$$

expressed in V/s, with the electrophysiological convention *outward (cation efflux) current positive*. Equation (2) therefore adds $-I_{\text{ion_cm}}$ to the PDE source; the external stimulus `externalStimulusCurrent` is stored in A/m^3 and divided by χC_m inside the solver (see `createFields.H` and the source-vector assembly in the main solver loop). Throughout the rest of this report the symbol I_{ion} is used as a shorthand for $I_{\text{ion_cm}}$; no further rescaling is applied between the cell model and the linear system.

2.3 External stimulus

$$I_{\text{ext}}(\mathbf{x}, t) = \begin{cases} I_0 & \mathbf{x} \in \Omega_{\text{stim}}^{(b)}, \quad t \in [t_0^{(b)}, t_0^{(b)} + \tau], \\ 0 & \text{otherwise,} \end{cases} \quad (4)$$

with Niederer-benchmark defaults $I_0 = 50\,000 \text{ A m}^{-3}$ and $\tau = 2 \text{ ms}$.

3 Time discretisation: the θ -scheme

Time is advanced from t^n to $t^{n+1} = t^n + \Delta t$ by

$$\frac{V^{n+1} - V^n}{\Delta t} = \theta L V^{n+1} + (1 - \theta) L V^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n. \quad (5)$$

<code>implicitScheme</code>	θ	Scheme
<code>backwardEuler</code>	1	First-order, L-stable
<code>crankNicolson</code>	1/2	Second-order, A-stable

After dividing by Δt the implementation forms

$$[M/\Delta t - \theta L] V^{n+1} = [M/\Delta t + (1 - \theta) L] V^n + \theta \mathbf{s}^{n+1} + (1 - \theta) \mathbf{s}^n, \quad (6)$$

with the two Eigen `SparseMatrix<scalar, RowMajor>` operators

$$A = M/\Delta t - \theta L, \quad B = M/\Delta t + (1 - \theta) L, \quad L = K/(\chi C_m). \quad (7)$$

A and B are rebuilt at every time step. When $\theta = 1$ (BE) the explicit half of the diffusion vanishes and $B = M/\Delta t$.

Old-time source $\mathbf{s}^n$. The vector $\mathbf{s}^n$ used in the RHS is *not* carried over verbatim from the previous step. At the start of each step the TNNP ODE is integrated once from $\mathbf{s}^n$ with V_m^n frozen, producing I_{ion}^n and hence $\mathbf{s}^n$; this is the *step-entry* solve. Subsequent Picard sweeps (Section 5) only overwrite the new-time iterate $\mathbf{s}(I_{\text{ion}}(\mathbf{x}_k))$ and leave the step-entry $\mathbf{s}^n$ in place. Across time steps the consistency of $\mathbf{s}^n$ relies on the ionic state vector $\mathbf{s}_j^n$ accepted at the end of the previous step being the same one used to recompute I_{ion}^n here.

4 Spatial discretisation

4.1 Standard orthogonal FV Laplacian

`assembleStandardOrthogonalStiffnessMatrix` produces the classical 5/7-point Laplacian. Per internal face f ,

$$a_f = \frac{|\mathbf{S}_f| (\mathbf{n}_f \cdot \mathbf{D}_f \cdot \hat{\mathbf{d}}_f)}{|\mathbf{d}_f|}, \quad \mathbf{D}_f = \frac{1}{2} (\mathbf{D}_{\text{own}} + \mathbf{D}_{\text{nei}}), \quad (8)$$

yielding the four standard contributions to K. `zeroGradient` and `empty` patches contribute nothing; `fixedValue` patches contribute only to the owner diagonal.

4.2 High-order LRE diffusion

When `useHighOrder_Vm = true` and $d > 1$, `assembleHighOrderStiffnessMatrix` integrates the flux on a face quadrature with reconstructed gradients:

$$\Phi_f = \sum_{q \in \mathcal{Q}_f} w_q |\mathbf{S}_f| \mathbf{n}_f \cdot \mathbf{D}_f \cdot \left(\sum_{j \in \mathcal{S}_f} \mathbf{g}_{f,q,j} \mathbf{V}_j \right), \quad (9)$$

with $\mathcal{S}_f$ the LRE face stencil and $\mathbf{g}_{f,q,j} = \text{QRGradFaceGPCoeffs}$ [4].

Boundary faces in the high-order branch. The face-quadrature integration above is applied only to *internal* faces. Boundary faces are delegated to the standard-FV path described in §4.1: `zeroGradient` and `empty` patches contribute zero flux, and `fixedValue` patches contribute the one-sided owner-only stencil to $\mathbf{K}$. The high-order LRE stencil is therefore strictly internal and the formal spatial order is expected to drop to that of the standard branch in the first layer of cells adjacent to a non-empty patch. Provided the activation front is interior to the domain (the case in the 2D benchmark used in Section 12) this near-boundary order drop is invisible to the reported activation-time metric.

4.3 Mass matrices

Two options selectable via `massMatrix`:

Lumped. `assembleDiagonalMassMatrix` writes $\mathbf{M} = \mathbf{I}$.¹

Consistent high-order. `assembleConsistentMassMatrixH0` fills

$$\mathbf{M}[i, j] = \sum_{q \in \mathcal{Q}_i} \frac{w_q}{\sum_{q'} w_{q'}} \phi_j(\mathbf{x}_q), \quad (10)$$

where ϕ_j is the LRE Taylor-basis function of stencil cell j (value δ_{ij} + grad + Hessian + 3rd-order derivative).

4.4 High-order reconstruction at ionic quadrature points

If `useHighOrder_Iion = true` the ionic ODE is integrated at cell quadrature points; V_m is reconstructed there by the same Taylor expansion (`reconstructVmAtIionIntegrationPoints`). The ionic contribution to the cell average is then

$$I_{\text{ion},i} = \frac{\sum_{q \in \mathcal{Q}_i} w_q I_{\text{ion},q}}{\sum_{q \in \mathcal{Q}_i} w_q} \quad (\text{averageIionIntegrationPointsToCells}). \quad (11)$$

5 Nonlinear solver

The fully discrete equation is nonlinear in $\mathbf{V}^{n+1}$ through $I_{\text{ion}}(\mathbf{V}^{n+1}, \mathbf{s}^{n+1})$. The implemented closure is fixed-point Picard. The solver does not currently read a `nonlinearMethod` key and does not assemble a finite-difference diagonal approximation of $\partial I_{\text{ion}} / \partial V_m$; the `nonlinearIterations` entry only sets the number of Picard sweeps.

¹The identity is *not* the FE-style row-sum lumped mass. It is the cell-averaged FV mass: the semi-discrete equation has already been divided by the cell volume $|\Omega_i|$, so the cell-average operator reduces to a scalar identity. The cell volumes are absorbed into $\mathbf{K}$ (via the face-area factor $|\mathbf{S}_f|$) and into the source assembly. A reader coming from FE codes should not expect $\mathbf{M}[i, i]$ to scale with $|\Omega_i|$ here.

5.1 Picard iteration

Given an iterate $\mathbf{x}_k$, the next iterate is the unique solution of the linear system

$$\mathbf{A}\mathbf{x}_{k+1}^* = \mathbf{B}\mathbf{V}^n + \theta \mathbf{s}(I_{\text{ion}}(\mathbf{x}_k)) + (1 - \theta) \mathbf{s}^n, \quad (12)$$

followed by the assignment $\mathbf{x}_{k+1} = \mathbf{x}_{k+1}^*$. The ionic ODE is re-integrated from t^n to t^{n+1} at every sweep using $\mathbf{x}_k$ as the (frozen) input voltage.

5.2 Iteration budget and acceptance

The outer loop performs `max(nonlinearIterations, 1)` Picard sweeps per time step. There are no residual-based nonlinear stopping criteria in the current implementation: after the last sweep, the last voltage iterate, ionic current, and ionic states are accepted.

Consequence of `nonlinearIterations = 1`. All runs reported in Section 12 use one Picard sweep per step. With $k = 1$ the new-time source $\theta \mathbf{s}(I_{\text{ion}}(\mathbf{x}_1))$ is evaluated from the iterate $\mathbf{x}_1 = \mathbf{V}^n$ (the loop entry value), and no correction is applied before the voltage is accepted. The scheme is then *linearly implicit in V_m and explicitly lagged in I_{ion}* : diffusion is treated implicitly via $\mathbf{A}$, but the reaction term is effectively first-order accurate in Δt , *independently of θ* . The temporal error therefore tracks the linearisation, not the formal order of the θ -scheme; this is the dominant source of the Δt -sensitivity observed in Section 12. Restoring the $\theta = 1/2$ second-order behaviour requires at least one residual-converged Picard sweep, i.e. $k \geq 2$ in practice with a small stop tolerance on $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|$.

6 Linear solver: Eigen back-end

The wrapper `solveSparseSystem` drives either of two Eigen sparse solvers on the implicit operator:

linearSolver key	Eigen type	Kind
<code>SparseLU</code>	<code>Eigen::SparseLU</code>	Direct LU factorisation
<code>BiCGSTAB</code>	<code>Eigen::BiCGSTAB + IncompleteLUT</code>	Iterative

Both share the `tolerance` and `maxIterations` dictionary keys (the former is ignored by `SparseLU`). The Picard loop solves with the same implicit operator `AImplicit`; no nonlinear diagonal correction is added to the left-hand side.

7 Ionic ODE integration

For each cell or cell QP, the TNNP ODE (3) is advanced by the OpenFOAM adaptive integrator selected via the `solver` key in `timeIntegrationProperties`. Default `RKF45` [3]. The keys read from that file are:

Key	Meaning	Default
<code>solver</code>	ODE integrator	<code>RKF45</code>
<code>absTol</code>	Absolute tolerance per state	10^{-9}
<code>relTol</code>	Relative tolerance per state	10^{-6}
<code>maxSteps</code>	Cap on sub-steps per outer call	10^4
<code>initialODEStep</code>	Initial sub-step (ms)	10^{-5}

8 TNNP numerical protection

The TNNP 2004 cell model contains several mathematical features that become numerical traps when the state variables are driven outside their physiological window by the outer solver: gate variables that overshoot $[0, 1]$ make the rate-balance equations stiff or unphysical; intracellular concentrations that touch zero force $\log(\cdot)$ / $\sqrt{\cdot}$ / $\text{pow}(\cdot)$ to overflow; very large $|V|$ saturates the $\exp(\cdot)$ tails of the α/β rate constants; and the L -type calcium current i_{CaL} contains a removable singularity at $V = 0$ (a $V/(\exp(V/RT) - 1)$ factor). The TNNP wrapper provides a guard, `protectState(state, t, qp, phase)`, whose job is to keep the state vector inside that physiological window at the boundaries of every external call into the cell-model kernel. The guard is intentionally minimal: it does *not* fix the cell model, it only prevents one bad time-step from cascading into NaN/Inf and corrupting the rest of the simulation.

8.1 Master switch and configuration

Key	Type	Default
<code>TNNPNumericalProtection</code>	Switch	<code>false</code>
<code>TNNPCLampGates</code>	Switch	<code>true</code>
<code>TNNPCLampVmForRates</code>	Switch	<code>true</code>
<code>TNNPLogNumericalProtection</code>	Switch	<code>true</code>
<code>TNNPConcentrationFloor</code>	scalar (mM)	10^{-12}
<code>TNNPVMInForRates</code>	scalar (mV)	-200
<code>TNNPVMMaxForRates</code>	scalar (mV)	$+100$
<code>TNNPVoltageZeroEps</code>	scalar (mV)	10^{-8}
<code>TNNPMaxProtectionLogEntries</code>	label	10^6

The master switch is `TNNPNumericalProtection`. When it is `false`, `protectState` returns immediately and *none* of the sub-switches have any effect; in particular, `TNNPCLampGates` and `TNNPCLampVmForRates` default to `true` but are inert without the master switch. The default configuration is therefore *all protection off*; production runs typically need to set `TNNPNumericalProtection true` explicitly.

8.2 Phases at which the guard fires

The wrapper invokes the guard at four phases, distinguished by a string tag passed to `protectState` (used only for logging / accounting; see §8.6):

Phase tag	Call site	Acts on
<code>calculateCurrent</code>	entry of <code>calculateCurrent</code>	the state vector passed in for the algebraic I
<code>solveODE_start</code>	entry of <code>solveODE</code>	the integrator's working state vector at t^n
<code>solveODE_end</code>	exit of <code>solveODE</code>	the integrator's working state vector at t^{n+1}
<code>derivatives</code>	inside <code>derivatives()</code> , every RKF45 sub-step	a <i>copy</i> of the state vector (does not mutate y)

All four sites are gated by the single `TNNPNumericalProtection` flag. The `derivatives` site has one important asymmetry: it copies the integrator state and protects the copy, then feeds the copy into the rate kernel `TNNPcomputeRates`. The integrator's working vector y is never modified mid-step, so the protection only changes the value of the rates RKF45 sees, not the state it integrates.

8.3 Corrections and their order

When fired, `protectState` applies the following corrections, in this order, to the (possibly copied) state vector:

1. **Non-finite V :** if `state[V]` is NaN or Inf, it is reset to -86.2 mV (TNNP resting potential, by construction inside `[vmMin, vmMax]`).
2. **Voltage clamp for rates** (if `TNNPClampVmForRates`): V is clamped to `[TNNPVMInForRates, TNNPVMaxForRates]`. After step 1 this step is a no-op; it only fires when V was finite but outside the rate window.
3. **Avoid i_{CaL} singularity:** if $|V| < \text{TNNPVoltageZeroEps}$, V is pushed to $\pm \text{TNNPVoltageZeroEps}$ (sign preserved). This removes the $V/(\exp(V/RT) - 1)$ removable singularity in i_{CaL} at the symbolic level. Steps 1–2 may have moved V off zero already; this is the safety net for the case where V is finite, inside the rate window, but right on zero.
4. **Concentration floor:** each of K_i, Na_i, Ca_i, Ca_{SR} is replaced by `TNNPConcentrationFloor` (10^{-12} mM) if it is non-finite or below the floor. This prevents $\log(\cdot)$ and rational-exponent operations on near-zero concentrations from overflowing.
5. **Gate clamp** (if `TNNPClampGates`): the twelve gating variables $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ are clamped to $[0, 1]$ (with non-finite values pulled to 0). This is the most frequent correction in practice because adaptive ODE sub-stepping can overshoot the bounds by $\mathcal{O}(\text{relTol})$ between t^n and t^{n+1} .

Each step is idempotent and operates on the already-corrected state, so a single `protectState` call is enough — there is no inner fixed-point iteration. The ionic currents are computed from the corrected state after step 5.

8.4 Interaction with the RKF45 step controller

The `derivatives` call site introduces *non-smoothness* in the right-hand side seen by RKF45: clamping is C^0 at best (piecewise linear in the offending component), and the gate clamp is a saturation that flattens the Jacobian once the gate touches its bound. The RKF45 embedded error estimator implicitly assumes C^{q+1} smoothness of the RHS over the sub-step, so firing the guard inside `derivatives()` can inflate the local error estimate around clamp boundaries and trigger spurious sub-stepping. Because the integrator’s working state y is *not* mutated (§8.2), the practical effect is bounded: only the rates are perturbed, the state recovers as soon as the active gate leaves the clamp. Even so, a high count of `phase=derivatives` corrections in the summary file is a diagnostic signal that the cell ODE is being driven outside its valid regime — usually by a too-large Δt or a transient in V_m that the diffusion solver could not follow. The recommended response is to investigate the upstream cause; the `derivatives`-phase protection is a safety net, not a correction mechanism.

8.5 What the guard does not protect

The five corrections above address NaN/Inf propagation, gate excursion outside $[0, 1]$, the $V = 0$ singularity of i_{CaL} , and concentration underflow. They do *not* address the following remaining failure modes intrinsic to the TNNP cell model:

- Removable singularities in the α/β rate expressions of i_{Na} and i_{Kr} (notably the $1/(\exp(\cdot) - 1)$ factors at $V = -47.13$ mV and similar). These can be regularised *analytically* by L’Hôpital substitution near the singular point; the present wrapper leaves them to the adaptive integrator.
- Stiff blow-ups in the calcium-handling subsystem when Ca_{SR} approaches the floor or when fCa saturates close to 1. Clamping Ca_{SR} avoids overflow but does not regularise the Ca_i/Ca_{SR} ratio that drives the SR-release flux.
- Exponential overflow of the α/β tails for $|V| \gg \text{vmMax}$. Clamping V to the rate window is a workaround; the true fix is to ensure V never leaves the physiological window in the first place (i.e. solve the PDE accurately).

In short: the guard turns a numerical disaster (NaN cascade) into a recoverable mis-evaluation. It is not a substitute for an adequately-resolved outer solver.

8.6 Logging and accounting

Each invocation of `correctStateComponent` records a triplet (`phase`, `variable`, `reason`) together with the offending value, its corrected value, and the simulation time. If `TNNPLogNumericalProtection` is true, the entries are accumulated in memory (capped by `TNNPMaxProtectionLogEntries`) and emitted on solver exit to the file

`TNNP_numericalProtection_summary.dat`

in the case directory. The summary groups corrections by the (`phase`, `variable`, `reason`) key, with: count, first occurrence time, the first old/corrected value pair, and the min/max/ $|\Delta|_{\max}$ over all corrections in the group. Typical diagnostic use:

- Many `Vm_below_minimum` at one specific cell ID $\Rightarrow$ suspect a Dirichlet BC pulling V outside the rate window.
- Many `concentration_below_floor` after a long simulation $\Rightarrow$ ODE tolerances too loose; tighten `relTol`.
- Many `phase=derivatives` entries during the upstroke $\Rightarrow \Delta t$ too large for the reaction stiffness; either reduce Δt or add Picard sweeps (§5).
- Any `nonfinite_V` or `nonfinite_gate` $\Rightarrow$ a real divergence already happened upstream; the guard contained the damage but the result is suspect.

8.7 Defaults vs. recommended settings

The defaults are intentionally conservative (master switch `false`) to keep the wrapper transparent in test configurations. For production and convergence studies the recommended setting is:

Use case	<code>TNNPNumericalProtection</code>	Notes
Verification / MMS	<code>false</code>	Avoids hiding solver bugs.
Production runs	<code>true</code>	Gate/ V clamps on, log on.
Stress / stability	<code>true</code>	Inspect the summary file as the primary signal.

All 2D results in Section 12 were produced with the master switch `true`; each result directory contains the corresponding `TNNP_numericalProtection_summary.dat`.

9 Activation tracking and benchmarks

Activation time per cell. First-crossing of V_{thr} in a step is recorded by linear interpolation:

$$w = \text{clip}\left(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n}, 0, 1\right), \quad t_{\text{act}} = t^n + w \Delta t. \quad (13)$$

The per-cell flag is cleared so re-crossings are ignored.

Early termination on P8. When `stopAfterPointActivation` = `true`, the cell nearest `stopActivationPoint` (P8 in the Niederer benchmark) is monitored, and the effective end time is set to $t_{\text{act}} + \text{stopDelayAfterActivation}$.

Diagonal sampling. `sampleIDW` performs inverse-distance-weighted interpolation at `nDiagonalSamples` points between `diagonalStart` and `diagonalEnd` with $k = \text{sampleNeighbours}$, $p = \text{samplePower}$.

10 Solver flow chart

The pseudo-code below summarises the per-time-step flow and shows the `run_convergence.py` constants / dict keys that control each branch.

Algorithm 1 High-level time-step loop of `highOrderElectroActivationFoamImplicit`.

```

1: Read dictionaries.
2: Assemble Eigen SparseMatrixs: M (massMatrix), K (useHighOrder.Vm), L = K/( $\chi C_m$ ).
3:  $V^0 \leftarrow -84$  mV (initial condition).
4: while  $t^n < \text{END\_TIME}$  do
5:    $\Delta t \leftarrow \text{DT\_VALUES}$  (current sweep value).
6:   Build  $A = M/\Delta t - \theta L$ ,  $B = M/\Delta t + (1 - \theta) L$  (implicitScheme  $\rightarrow \theta$ ).
7:   Apply stimulus to  $I_{\text{ext}}$  (stimulusProtocol).
8:    $x_0 \leftarrow V^n$ .
9:   for  $k = 0, 1, \dots, \text{IMPLICIT\_NONLINEAR\_ITERATIONS}-1$  do
10:    Reconstruct  $V_m$  at quadrature points if useHighOrder.Iion.
11:    Integrate TNNP ODE with  $x_k$  (ODE_REL_TOL, ODE_ABS_TOL, solver=RKF45).
12:    Build RHS:  $r_k = B V^n + \theta s(I_{\text{ion}}(x_k)) + (1 - \theta) s^n$ .
13:    Solve  $A V_* = r_k$  with Eigen (linearSolver, tolerance, maxIterations).
14:     $x_{k+1} \leftarrow V_*$ .
15:   end for
16:   Accept the last Picard iterate, ionic current, and ionic states.
17:   Record activation times (activationThreshold); P8 stop check (stopAfterPointActivation,
   stopDelayAfterActivation).
18:    $t^{n+1} \leftarrow t^n + \Delta t$ .
19: end while
20: Post-process: IDW sampling at P8 and diagonal (nDiagonalSamples, sampleNeighbours, samplePower).

```

Legend. Blue = Python constant set in `run_convergence.py`; green = OpenFOAM dictionary key read by the solver.

11 Driver script: `run_convergence.py`

11.1 Geometry and parameter sweep

Python constant	OpenFOAM key	Meaning
DIMENSIONS	—	List of dimensions to sweep (2D, 3D).
DX_VALUES_MM	<code>blockMeshDict</code> cell counts	Uniform cell size in millimetres.
DT_VALUES	<code>controlDict.deltaT</code>	Outer time-step sizes (seconds).
END_TIME	<code>controlDict.endTime</code>	End time (s).
CFL	<code>timeIntegrationProperties.CFL</code>	Informational reference; not enforced (the solver is implicit).
ACTIVE_TIME_SCHEMES	<code>spatialIntegrationProperties.timeSchemes</code>	Implicit schemes (<code>backwardEuler</code> , <code>crankNicolson</code>).
ACTIVE_MASS_MATRICES	<code>spatialIntegrationProperties.massMatrices</code>	Implicit matrices.
ACTIVE_HO_PAIRS_Vm_States	<code>useHighOrder.Vm</code> <code>useHighOrder.Iion</code> <code>LRECoeffs_*</code>	/ Tuples (<code>Vm-level</code> , <code>Iion-level</code>); + level $\in \{N0, p1, p2, p3\}$.
ACTIVE_STATE_ODE_SOLVERS	<code>timeIntegrationProperties.solvers</code>	ODE integrator names (RKF45, ...).
SIGMA_X, SIGMA_Y, SIGMA_Z	<code>cardiacProperties</code>	Diagonal of D .
CHI	<code>cardiacProperties.chi</code>	Surface-to-volume ratio.
CM	<code>cardiacProperties.Cm</code>	Membrane capacitance.

11.2 Nonlinear and linear solver

Python constant	OpenFOAM key	Meaning
IMPLICIT_NONLINEAR_ITERATIONS	nonlinearIterations	Fixed Picard sweep count per step; at least one sweep is performed.
IMPLICIT_LINEAR_SOLVER	linearSolver	{SparseLU, BiCGSTAB}.
IMPLICIT_TOLERANCE	tolerance	BiCGSTAB relative tolerance.
IMPLICIT_MAX_ITERATIONS	maxIterations	BiCGSTAB iteration cap.

11.3 Inner ODE integrator (RK45)

Python constant	OpenFOAM key	Meaning
ODE_ABS_TOL	timeIntegrationProperties.absTol	Absolute tolerance per state.
ODE_REL_TOL	timeIntegrationProperties.relTol	Relative tolerance per state.
ODE_INITIAL_STEP	initialODEStep	Initial sub-step (ms).
ODE_MAX_STEPS	maxSteps	RKF45 sub-step cap.

11.4 Post-processing and run-control

Python constant	OpenFOAM key	Meaning
ACTIVATION_THRESHOLD	activationThreshold	V_{thr} (V).
N_DIAGONAL_SAMPLES	nDiagonalSamples	IDW samples.
SAMPLE_NEIGHBOURS	sampleNeighbours	k -NN for IDW.
SAMPLE_POWER	samplePower	Inverse-distance power.
STOP_AFTER_POINT_ACTIVATION	stopAfterPointActivation	Early-stop on P8.
STOP_DELAY_AFTER_ACTIVATION	stopDelayAfterActivation	Delay (s).
SAVE_LOGS_ON_SUCCESS	—	Python-only; keep logs after successful run.

12 2D Niederer-style benchmark — in-house convergence study

This section collects measured P8 activation times, wall-clock cost and peak memory.

Scope and disclaimer. The published Niederer et al. 2012 [2] benchmark is three-dimensional ($20 \times 7 \times 3$ mm anisotropic slab with fibre rotation). The case used here is a 2D reduction (20×8 mm, single cell in z , planar fibres) defined for code-development testing only and not reflected, to our knowledge, in any published reference. The numbers below should therefore be read as an *internal* self-consistency study — h - and Δt -refinement, plus the effect of the LRE p -levels — and not as a comparison with any external reference value.

Rows of the tables below are LRE p -levels for $V_m \times I_{\text{ion}}$ (NO = high-order disabled); columns are mesh sizes Δx in millimetres. Computational cost is the elapsed wall-clock time of the solver process reported by `/usr/bin/time -v`; peak memory is the maximum resident set size of the same process. The time-integration scheme and mass matrix are noted above each block when relevant. NC = no run data found for that combination.

In the reported runs, IMPLICIT_NONLINEAR_ITERATIONS=1, so the method behaves as a semi-implicit or linearly implicit scheme: diffusion is treated implicitly, while the ionic source is evaluated from a lagged Picard voltage and is not nonlinearly converged (see §5 for the consequences). All runs use the Crank–Nicolson scheme with the consistent high-order mass matrix.

12.1 Results for P8 activation

Table 5: P8 activation [ms].

Vm-Iion	Δt [s]	0.5	0.25	0.125	0.1	0.05	Vm-Iion	Δt [s]	0.5	0.25	0.125	0.1	0.05
NO-NO	10^{-3}	192.75	99.19	92.01	91.96	91.51	p1-NO	10^{-3}	0.00	109.58	93.32	92.33	91.81
	10^{-4}	161.05	69.88	53.92	52.21	50.29		10^{-4}	17.70	77.32	57.82	55.10	51.10
	10^{-5}	157.69	67.69	50.65	48.55	46.10		10^{-5}	17.33	74.83	54.52	51.58	47.05
	10^{-6}	157.37	67.46	50.32	48.18	45.69		10^{-6}	17.30	74.57	54.18	51.21	NC
NO-p1	10^{-3}	119.65	92.56	92.16	92.10	91.51	p1-p1	10^{-3}	145.47	100.72	94.02	92.70	91.80
	10^{-4}	96.18	61.22	52.11	51.29	50.20		10^{-4}	101.11	67.56	55.66	53.96	51.01
	10^{-5}	94.67	59.19	48.68	47.50	46.00		10^{-5}	99.06	65.26	52.26	50.31	46.93
	10^{-6}	94.51	58.99	48.34	47.13	45.59		10^{-6}	98.86	65.03	51.92	49.94	NC
NO-p2	10^{-3}	76.52	87.59	91.98	92.11	91.51	p1-p2	10^{-3}	82.36	91.27	93.30	93.14	91.81
	10^{-4}	37.79	46.32	48.97	49.42	49.93		10^{-4}	41.45	50.78	51.32	51.27	50.64
	10^{-5}	35.26	43.20	45.15	45.42	45.69		10^{-5}	38.95	47.69	47.65	47.41	46.49
	10^{-6}	35.04	42.92	44.79	45.04	NC		10^{-6}	38.74	47.39	47.29	NC	NC
NO-p3	10^{-3}	76.63	88.19	91.99	92.12	91.51	p1-p3	10^{-3}	83.96	91.90	93.30	93.15	NC
	10^{-4}	38.29	46.61	48.91	49.38	49.93		10^{-4}	43.36	51.34	51.27	51.23	NC
	10^{-5}	36.01	43.73	45.12	45.39	45.69		10^{-5}	41.20	48.40	47.62	47.38	NC
	10^{-6}	35.82	43.47	44.76	45.01	NC		10^{-6}	41.03	48.13	47.27	NC	NC
p2-NO	10^{-3}	0.00	127.89	97.27	93.22	92.29	p3-NO	10^{-3}	0.00	110.29	91.18	90.23	91.51
	10^{-4}	0.00	93.48	64.47	59.77	52.18		10^{-4}	189.23	76.54	58.33	55.11	50.64
	10^{-5}	0.00	90.68	61.15	56.52	48.35		10^{-5}	185.33	74.04	54.99	51.79	46.68
	10^{-6}	0.00	90.40	60.81	56.17	NC		10^{-6}	184.97	73.78	54.64	51.44	46.28
p2-p1	10^{-3}	192.75	114.03	96.97	93.64	92.27	p3-p1	10^{-3}	121.21	97.00	92.11	90.79	91.46
	10^{-4}	162.02	78.77	60.67	57.55	51.98		10^{-4}	72.53	65.88	55.21	53.43	50.52
	10^{-5}	159.19	76.35	57.44	54.19	48.08		10^{-5}	71.65	63.54	52.00	50.00	46.51
	10^{-6}	158.92	76.11	57.10	53.84	NC		10^{-6}	71.56	63.31	51.67	49.65	NC
p2-p2	10^{-3}	87.99	93.26	94.17	93.29	91.98	p3-p2	10^{-3}	75.08	87.35	91.44	91.29	91.31
	10^{-4}	46.23	53.99	52.87	52.50	51.07		10^{-4}	37.84	46.86	49.57	49.90	49.94
	10^{-5}	44.04	51.35	49.47	48.89	NC		10^{-5}	35.24	43.97	46.03	46.16	45.82
	10^{-6}	43.86	51.09	49.14	48.54	NC		10^{-6}	35.01	43.70	45.69	45.80	NC
p2-p3	10^{-3}	94.22	94.32	94.11	93.29	91.98	p3-p3	10^{-3}	71.20	88.09	91.41	91.32	91.31
	10^{-4}	52.60	55.56	53.00	52.53	51.07		10^{-4}	39.03	47.38	49.56	49.88	49.94
	10^{-5}	50.47	52.95	49.65	48.94	47.03		10^{-5}	36.80	44.73	46.06	46.16	45.82
	10^{-6}	50.29	52.70	49.33	NC	NC		10^{-6}	36.61	44.48	45.73	45.80	NC

12.2 Results for Wall-clock time

Table 6: Wall-clock time [min].

Vm-Iion	Δt [s]	0.5	0.25	0.125	0.1	0.05	Vm-Iion	Δt [s]	0.5	0.25	0.125	0.1	0.05
NO-NO	10^{-3}	1	2	6	10	39	p1-NO	10^{-3}	1	2	6	9	41
	10^{-4}	1	1	4	7	25		10^{-4}	1	2	6	8	35
	10^{-5}	2	3	8	13	52		10^{-5}	3	4	17	23	109
	10^{-6}	9	16	49	78	284		10^{-6}	19	30	134	155	NC
NO-p1	10^{-3}	1	3	13	19	78	p1-p1	10^{-3}	1	3	13	18	83
	10^{-4}	1	2	8	12	48		10^{-4}	1	3	11	13	64
	10^{-5}	2	5	16	24	90		10^{-5}	2	6	25	32	157
	10^{-6}	10	26	88	138	513		10^{-6}	17	39	146	203	NC
NO-p2	10^{-3}	2	8	39	54	242	p1-p2	10^{-3}	2	10	36	58	266
	10^{-4}	1	5	24	33	145		10^{-4}	1	6	23	38	180
	10^{-5}	2	9	44	60	243		10^{-5}	2	23	46	78	442
	10^{-6}	11	49	212	402	NC		10^{-6}	13	87	280	NC	NC
NO-p3	10^{-3}	2	11	52	74	276	p1-p3	10^{-3}	3	13	47	84	57
	10^{-4}	1	7	31	45	178		10^{-4}	2	8	29	52	NC
	10^{-5}	3	12	55	84	336		10^{-5}	3	18	58	100	NC
	10^{-6}	14	66	289	510	NC		10^{-6}	18	123	359	NC	NC

Vm-Ion	Δt [s]	0.5	0.25	0.125	0.1	0.05	Vm-Ion	Δt [s]	0.5	0.25	0.125	0.1	0.05
p2-NO	10^{-3}	1	2	7	11	46	p3-NO	10^{-3}	1	2	7	11	44
	10^{-4}	2	2	7	11	60		10^{-4}	1	2	8	12	52
	10^{-5}	4	8	25	36	174		10^{-5}	4	9	31	53	207
	10^{-6}	27	67	220	328	NC		10^{-6}	30	74	254	415	1845
p2-p1	10^{-3}	2	4	13	20	118	p3-p1	10^{-3}	1	4	13	20	81
	10^{-4}	2	3	11	16	81		10^{-4}	1	3	10	18	78
	10^{-5}	4	10	33	46	210		10^{-5}	3	10	35	61	268
	10^{-6}	30	72	276	394	NC		10^{-6}	23	80	287	450	NC
p2-p2	10^{-3}	2	9	40	85	246	p3-p2	10^{-3}	2	8	38	58	237
	10^{-4}	1	6	27	50	171		10^{-4}	1	5	24	40	172
	10^{-5}	3	14	58	99	NC		10^{-5}	3	13	57	99	415
	10^{-6}	17	89	446	730	NC		10^{-6}	15	85	411	704	NC
p2-p3	10^{-3}	3	13	53	81	309	p3-p3	10^{-3}	2	11	50	88	372
	10^{-4}	2	9	35	52	277		10^{-4}	2	7	34	54	233
	10^{-5}	4	19	69	109	597		10^{-5}	3	17	78	133	563
	10^{-6}	25	123	521	NC	NC		10^{-6}	19	106	469	892	NC

12.3 Results for Peak RSS

Table 7: Peak RSS [MiB].

Vm-Ion	Δt [s]	0.5	0.25	0.125	0.1	0.05	Vm-Ion	Δt [s]	0.5	0.25	0.125	0.1	0.05
NO-NO	10^{-3}	148	154	178	196	362	p1-NO	10^{-3}	152	166	233	279	687
	10^{-4}	149	154	178	196	362		10^{-4}	152	166	234	279	687
	10^{-5}	149	155	178	196	362		10^{-5}	151	167	233	279	673
	10^{-6}	150	154	178	195	360		10^{-6}	152	166	231	280	NC
NO-p1	10^{-3}	150	158	195	226	482	p1-p1	10^{-3}	153	170	248	308	805
	10^{-4}	150	158	195	226	481		10^{-4}	153	170	247	308	801
	10^{-5}	150	158	195	226	481		10^{-5}	153	170	246	311	785
	10^{-6}	150	158	195	225	480		10^{-6}	153	170	247	310	NC
NO-p2	10^{-3}	155	176	269	341	946	p1-p2	10^{-3}	157	188	321	423	1250
	10^{-4}	155	176	269	341	946		10^{-4}	157	187	320	422	1248
	10^{-5}	155	176	269	342	945		10^{-5}	157	188	318	420	1230
	10^{-6}	155	176	269	341	NC		10^{-6}	157	188	318	NC	NC
NO-p3	10^{-3}	157	184	306	400	1177	p1-p3	10^{-3}	159	195	358	481	1480
	10^{-4}	158	184	306	399	1177		10^{-4}	160	195	357	479	NC
	10^{-5}	157	185	306	399	1177		10^{-5}	160	195	356	477	NC
	10^{-6}	157	184	306	399	NC		10^{-6}	160	195	355	NC	NC
p2-NO	10^{-3}	152	169	260	329	852	p3-NO	10^{-3}	155	192	345	466	1370
	10^{-4}	152	169	258	327	882		10^{-4}	155	191	344	463	1371
	10^{-5}	153	170	259	323	853		10^{-5}	155	191	342	461	1371
	10^{-6}	153	170	261	324	NC		10^{-6}	155	192	342	461	1371
p2-p1	10^{-3}	154	174	278	360	975	p3-p1	10^{-3}	157	196	361	485	1465
	10^{-4}	154	174	277	357	969		10^{-4}	157	196	365	482	1457
	10^{-5}	154	174	276	352	949		10^{-5}	157	196	362	482	1482
	10^{-6}	154	175	277	355	NC		10^{-6}	157	197	362	482	NC
p2-p2	10^{-3}	158	193	348	473	1429	p3-p2	10^{-3}	161	214	437	608	1923
	10^{-4}	158	192	345	470	1423		10^{-4}	161	213	434	605	1912
	10^{-5}	158	193	345	466	NC		10^{-5}	162	212	433	603	1893
	10^{-6}	158	192	344	466	NC		10^{-6}	161	213	432	602	NC
p2-p3	10^{-3}	160	202	390	531	1661	p3-p3	10^{-3}	163	223	474	666	2155
	10^{-4}	160	202	387	528	1654		10^{-4}	163	222	472	663	2142
	10^{-5}	161	202	387	524	1625		10^{-5}	164	222	470	660	2123
	10^{-6}	160	202	386	NC	NC		10^{-6}	164	222	469	658	NC

12.4 Discussion of the 2D results

The four scalar refinement axes — mesh size Δx , time step Δt , LRE order on the diffusion (`useHighOrder_Vm`), and LRE order on the ionic source (`useHighOrder_Iion`) — can be read off the tables in pairs. The observations below use the P_8 activation time $t_{\text{act}}^{P_8}$ as the single scalar output (in ms).

h -refinement at fixed Δt (low order). For the pair (N0, N0) with Crank–Nicolson and consistent mass matrix, holding $\Delta t = 10^{-3}$ s and refining the mesh gives, from $\Delta x = 0.5$ to 0.05 mm, $t_{\text{act}}^{P_8} = 192.75, 99.19, 92.01, 91.96, 91.51$ ms. Only the two coarsest meshes are clearly under-resolved; from $\Delta x \leq 0.125$ mm the value is mesh-stable to within ~ 0.5 ms. At this Δt the front is well-resolved in space even with the low-order scheme.

Δt -refinement at fixed h (low order). On the same low-order pair at $\Delta x = 0.05$ mm, refining Δt from 10^{-3} to 10^{-6} s gives 91.51, 50.29, 46.10, 45.69 ms. A one-order-of-magnitude reduction in Δt between 10^{-3} and 10^{-4} *halves* the activation time; the following decade ($10^{-4} \rightarrow 10^{-5}$) moves it by another ~ 4 ms, and the last decade ($10^{-5} \rightarrow 10^{-6}$) by only ~ 0.4 ms. This is not the $\theta = 1/2$ asymptotic order of Crank–Nicolson; it is the linearisation-error signature of `nonlinearIterations` = 1 (§5). With a single Picard sweep, Δt is bounded above by the reaction stiffness rather than by the diffusion stability or the formal θ -order. Practical mesh-converged behaviour for the low-order pair therefore requires $\Delta t \lesssim 10^{-5}$ s in this benchmark.

Effect of the LRE p -level on the source. Varying only the second slot at the same $(\Delta x, \Delta t)$ — e.g. (N0, N0) vs. (N0, p2) — isolates the high-order ionic integration. At $\Delta x = 0.5$ mm and $\Delta t = 10^{-5}$ s, $t_{\text{act}}^{P_8}$ moves from 157.69 to 35.26 ms; at $\Delta x = 0.25$ mm from 67.69 to 43.20 ms. The cell-averaged source $\overline{I_{\text{ion}}}$ computed at LRE cell quadrature points differs from $I_{\text{ion}}(\overline{V_m})$ wherever V_m has non-trivial curvature across the cell — exactly during the upstroke — and the difference is large on coarse meshes. The size of the (N0, N0) vs. (N0, p2) gap is a useful proxy for under-resolution of the front at that $(\Delta x, \Delta t)$: when the two configurations agree, the cell is small enough that piecewise-constant V_m inside the cell is a good approximation.

Effect of the LRE p -level on the diffusion. With (p3, p3) at $\Delta x = 0.125$ mm, $\Delta t = 10^{-6}$ s, $t_{\text{act}}^{P_8} = 45.73$ ms; at $\Delta x = 0.5$ mm with the same Δt , the value is 36.61 ms. Notice the trend is *non-monotone* in Δx for the (p3, p3) pair: the coarsest meshes give the *lowest* activation times, with the values increasing and plateauing as Δx shrinks (cf. also (N0, p2) which behaves similarly). This is the signature of an LRE stencil that, on a very coarse mesh, sees a substantial part of the front within its support and effectively over-smooths the source; on a finer mesh the stencil is local enough that the average becomes faithful again. The values at $\Delta x = 0.1$ mm, $\Delta t = 10^{-6}$ s lie in $[45.04, 45.80]$ ms across all four high-order configurations we inspected — the best mutually-consistent estimate of $t_{\text{act}}^{P_8}$ available in the current data set.

Cost. Peak RSS grows roughly linearly in the number of cells N_c (the Eigen sparse matrices dominate memory). Wall time grows roughly as $N_c/\Delta t$ for fixed end-time, with the LRE high-order branches paying a constant-factor overhead from the larger stencil (mass-matrix assembly and the per-quadrature-point ionic ODE). At $\Delta t = 10^{-3}$ s the cost ratio between (p3, p3) and (N0, N0) at fixed Δx is much larger than the accuracy ratio at that Δt — but the balance flips at $\Delta t = 10^{-5}$ s, where (p3, p3) on a coarse mesh already matches the low-order configuration on the finest mesh.

12.5 Conclusions on the 2D study

1. The dominant numerical error in the reported configuration is *the linearisation error from a single Picard sweep*. This is visible in the strong Δt -dependence of the low-order pair ($192 \rightarrow 46$ ms across the Δt sweep on the finest mesh) and the much weaker Δt -dependence of the high-order

pairs. Until the Picard loop is closed (either by adding sweeps with a residual stop, or by adding a $\partial I_{\text{ion}}/\partial V_m$ Jacobian diagonal correction), the formal $\theta = 1/2$ second-order behaviour will not be recovered.

2. LRE high-order integration of the ionic source (p2 / p3) buys substantial accuracy on coarse meshes and partially compensates for under-resolution of the activation front. It is more cost-effective than h -refinement at the same target accuracy, *for the activation-time metric*.
3. The high-order configurations agree among themselves to within $\lesssim 1$ ms at $\Delta x = 0.1$ mm and $\Delta t = 10^{-6}$ s, which we take as the internal-consistency floor of the present data. We do not claim this represents the “true” value of t_{act}^{P8} for this 2D problem — there is, as noted, no published reference to compare against.
4. The natural next step is to enable a second Picard sweep (or, equivalently, a tolerance-based stop on the Picard residual) and re-run the Δt sweep. We expect the Δt -sensitivity above 10^{-4} s to collapse and the $\theta = 1/2$ second-order behaviour to become visible.

13 3D Niederer-style benchmark

The 3D sweep follows the published Niederer slab geometry ($20 \times 3 \times 7$ mm, $40 \times 6 \times 14$ to $200 \times 30 \times 70$ cells for $\Delta x \in \{0.5, 0.2, 0.1\}$ mm) and shares the same activation tracker as the 2D study. All 3D runs use Crank–Nicolson with the consistent high-order mass matrix, the TNNP cell model with default `numericalProtection`, and a single requested $\Delta t_{\text{req}} = 5 \cdot 10^{-5}$ s; the implicit scheme integrates with that step at every mesh size. Only three LRE pairs are currently available (NO/NO, NO/p1, p3/p3), and not every pair has every mesh: the coarsest mesh ($\Delta x = 0.5$ mm) exists for all three, the medium one ($\Delta x = 0.2$ mm) only for the two low-order pairs, and the finest ($\Delta x = 0.1$ mm) only for the fully low-order pair. The tables below report what is present; missing combinations are marked NC. The runs are stored under `highOrderNiedererEtAl2012Implicit/results/example0/3D/`.

Δx [mm]	$40 \times 6 \times 14$ (0.5)	$100 \times 15 \times 35$ (0.2)	$200 \times 30 \times 70$ (0.1)
N_c	3360	52 500	420 000

13.1 P8 activation time at $\Delta t = 5 \cdot 10^{-5}$ s

Table 8: 3D P8 activation time [ms]. Rows are LRE p-levels for $V_m \times I_{\text{ion}}$; columns are mesh sizes Δx (mm). All runs use `crankNicolson` and the consistent mass matrix.

V_m	I_{ion}	0.5	0.2	0.1
NO	NO	142.70	55.55	47.31
NO	p1	38.01	44.60	NC
p3	p3	34.50	NC	NC

13.2 Wall-clock cost per configuration (minutes)

Table 9: 3D solver wall-clock time [min] (elapsed wall time reported by `/usr/bin/time -v`, rounded to integer minutes).

V_m	I_{ion}	0.5	0.2	0.1
NO	NO	4	41	263
NO	p1	14	232	NC
p3	p3	151	NC	NC

13.3 Peak resident set size per configuration (MiB)

Table 10: 3D peak resident set size [MiB] (maximum physical RAM reported by `/usr/bin/time -v`).

Vm	Iion	0.5	0.2	0.1
NO	NO	157	334	1666
NO	p1	219	1383	NC
p3	p3	1653	NC	NC

13.4 Discussion of the 3D results

At the coarsest mesh ($\Delta x = 0.5$ mm) the (NO, NO) pair gives $t_{\text{act}}^{P8} \approx 142.7$ ms. Switching the ionic quadrature to p1 on the same coarse mesh collapses it to 38.0 ms, and the fully high-order pair (p3, p3) reaches 34.5 ms — consistent with the same qualitative pattern observed in the explicit 3D sweep: high-order on the source is the single biggest accuracy lever on coarse meshes. Mesh refinement of the low-order pair brings t_{act}^{P8} from 142.7 to 55.6 to 47.3 ms; the (NO, p1) row goes from 38.0 to 44.6 ms on the only two meshes where it is currently available.

Wall time grows roughly linearly in the number of cells (low-order pair: $4 \rightarrow 41 \rightarrow 263$ min for N_c multiplying by ~ 15 each refinement, with the per-step linear-solver cost growing slightly super-linearly because of conditioning). The high-order branches pay a large constant-factor overhead on top of that: at $\Delta x = 0.5$ mm the (p3, p3) cost is already $\sim 38\times$ the (NO, NO) cost on the same mesh. Peak RSS scales close to linearly with N_c for the low-order pair (157, 334, 1666 MiB) and adds the LRE-stencil overhead for the high-order pairs.

The 3D sweep is therefore consistent with the 2D study but much sparser: it confirms that high-order on the source is the cost-effective accuracy lever, but the dataset is too thin to draw quantitative conclusions about the asymptotic t_{act}^{P8} of this 3D problem.

A Dimension table

Field	OpenFOAM dimensionSet	SI
Vm	[1, 2, -3, 0, 0, -1, 0]	volt
Iion	dimVoltage/dimTime	V s^{-1}
externalStimulusCurrent	dimCurrent/dimVolume	A m^{-3}
conductivity	dimCurrent^2 dimTime^3 / (dimMass dimVolume)	S m^{-1}
chi	dimless/dimLength	m^{-1}
Cm	dimCurrent dimTime / (dimVoltage dimArea)	F m^{-2}

References

- [1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, *AJP – Heart and Circulatory Physiology* **286** (2004) H1573–H1589.
- [2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, *Prog. Biophys. Mol. Biol.* **104** (2012) 22–48.
- [3] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.
- [4] P. Cardiff, A. Karač, P. De Jaeger, H. Jasak, J. Nagy, A. Ivanković, Ž. Tuković, *An open-source finite volume toolbox for solid mechanics and fluid–solid interaction simulations*, arXiv:1808.10736 (2018). Provides the LRE high-order reconstruction library (LRE.H) used in §4.1 et seq.